

Программирование на С

Лекция 4

Поток управления

Проблема

До сих пор все инструкции выполнялись последовательно.

```
A -> B -> C -> D
```

Иногда программа должна выполнять разные действия в зависимости от значения переменной.

Например:

- разрешить или запретить доступ;
- вывести предупреждение;
- обработать ошибку;
- пропустить ненужные действия.

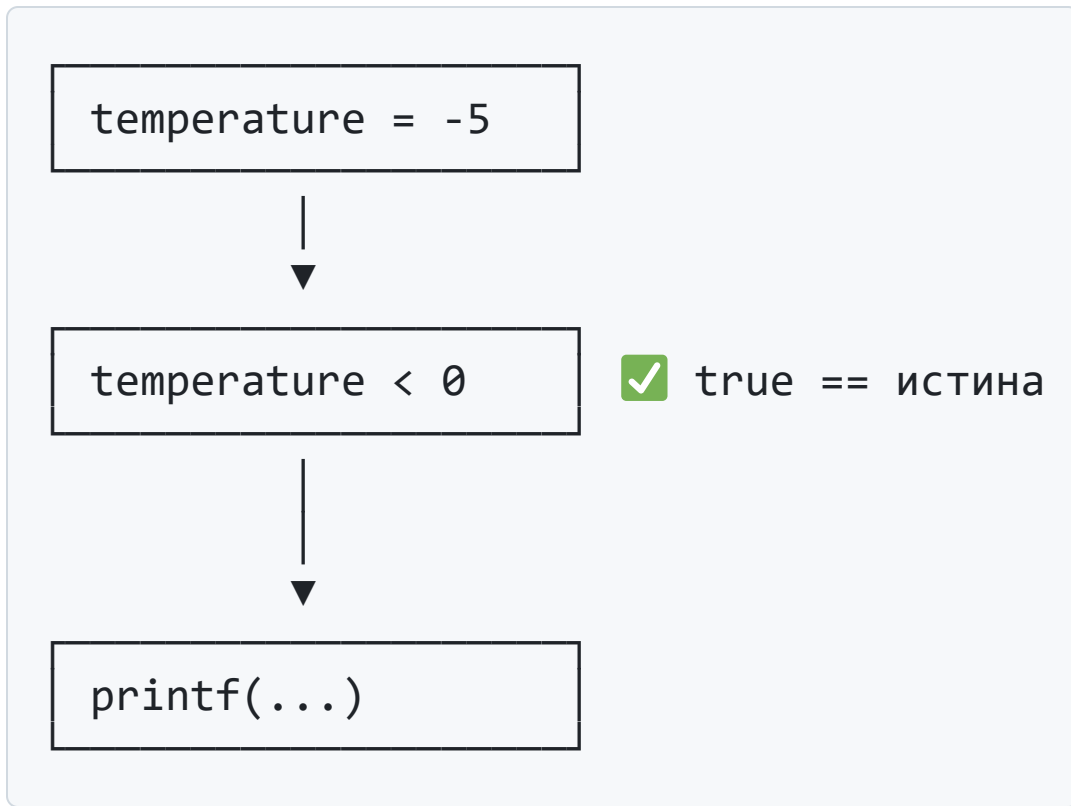
Оператор `if`

```
if (temperature < 0)
{
    printf("Warning: freezing\n");
}
```

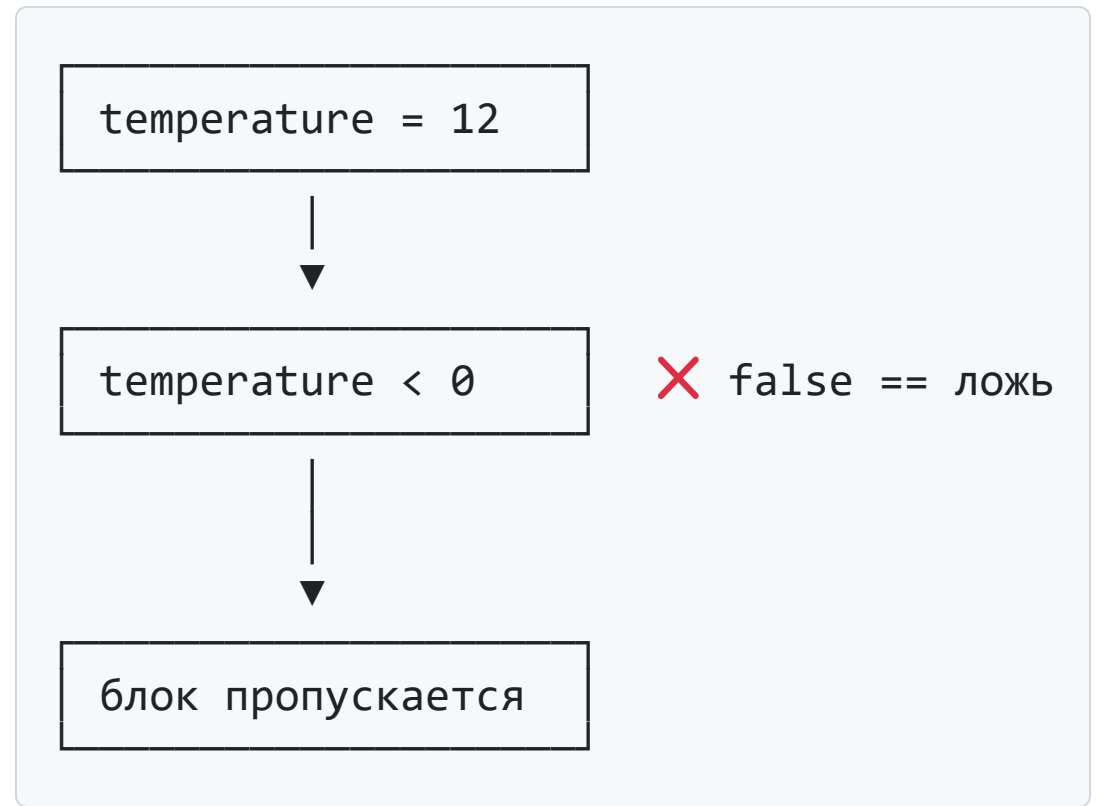
Если условие истинно, выполняется блок кода.

Как работает `if`

Условие истинно



Условие ложно



Операторы сравнения

Оператор	Значение
==	равно
!=	не равно
<	меньше
<=	меньше или равно
>	больше
>=	больше или равно

Оператор `else`

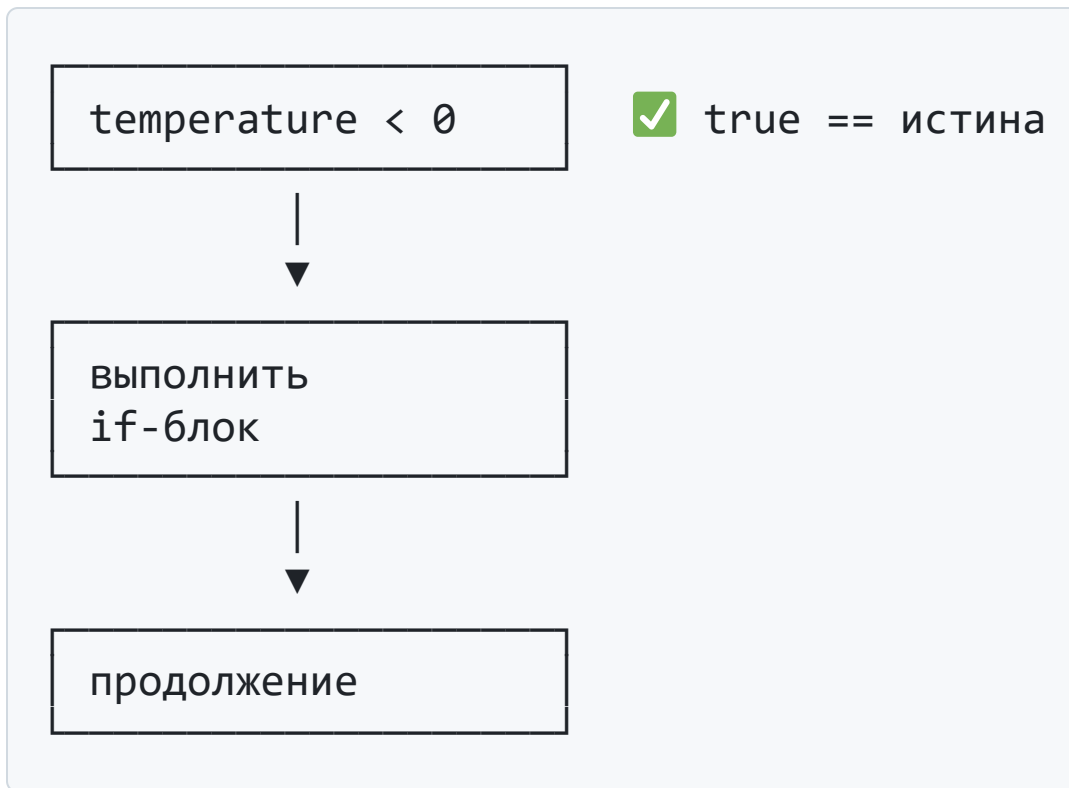
```
int temperature = 18;

if (temperature < 0)
{
    printf("Freezing\n");
}
else
{
    printf("Above zero\n");
}
```

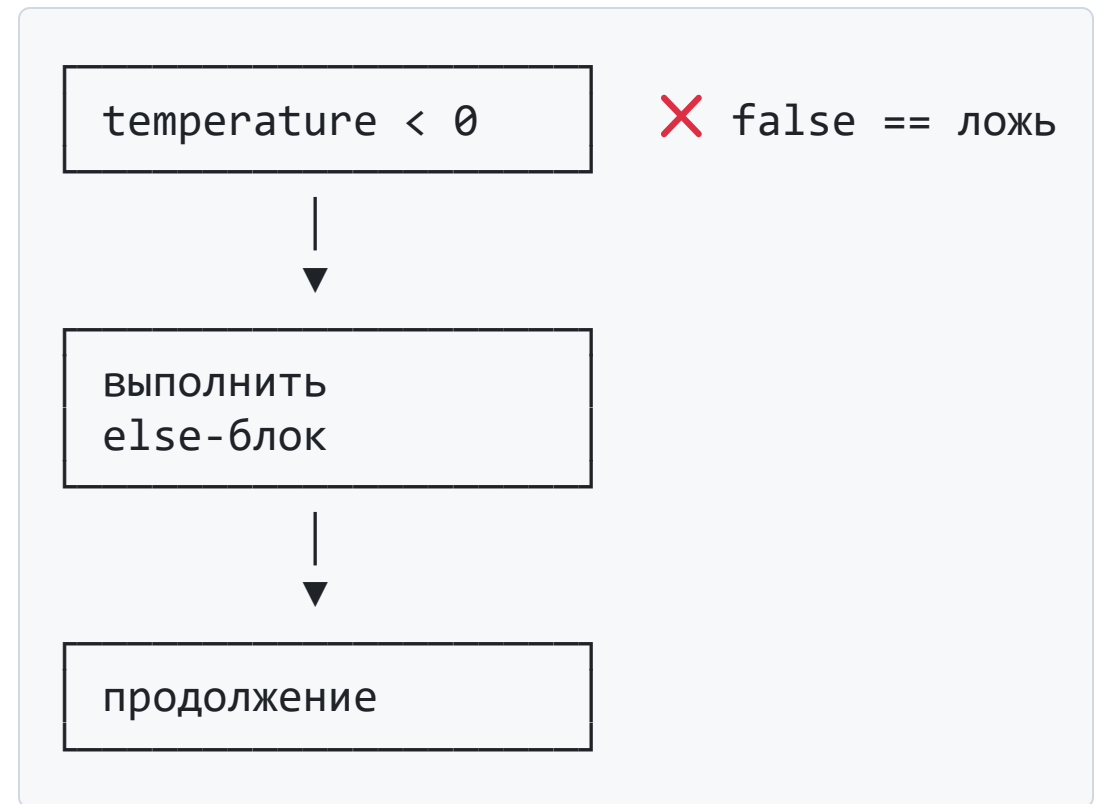
Если условие ложно, выполняется блок после `else` .

Выполняется только одна ветвь

Условие истинно



Условие ложно



После выполнения одной из ветвей программа продолжает работу.

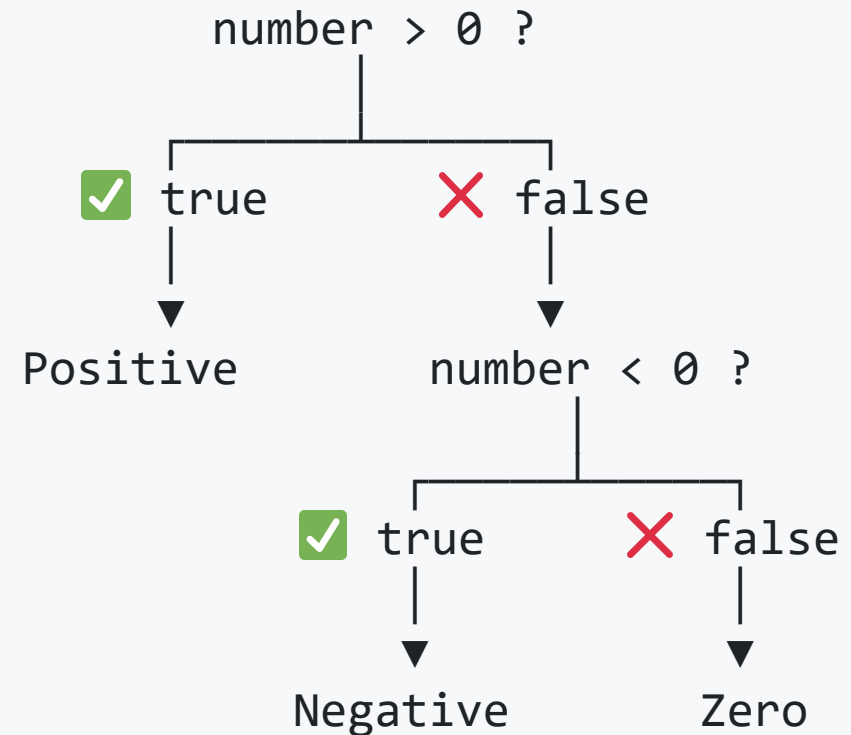
Иногда двух вариантов недостаточно

Например, нужно определить знак числа.

- число больше нуля;
- число меньше нуля;
- число равно нулю.

Вложенный if

```
if (number > 0)
{
    printf("Positive\n");
}
else
{
    if (number < 0)
    {
        printf("Negative\n");
    }
    else
    {
        printf("Zero\n");
    }
}
```



Внутри блока можно писать любые инструкции, в том числе другой if .

Оператор `else if`

Такая запись встречается очень часто, поэтому в языке `C` существует более удобная форма.

```
if (number > 0)
{
    printf("Positive\n");
}
else if (number < 0)
{
    printf("Negative\n");
}
else
{
    printf("Zero\n");
}
```

`else if` — это сокращённая запись вложенного `if`.

Фигурные скобки

Если после `if` выполняется только одна инструкция, фигурные скобки можно не писать.

✓ Можно

```
if (temperature < 0)
    printf("Freezing\n");
```

✓ Тоже правильно

```
if (temperature < 0)
{
    printf("Freezing\n");
}
```

Всегда следует использовать фигурные скобки.

Почему лучше всегда писать скобки?

✗ Легко ошибиться

```
if (temperature < 0)
    printf("Freezing\n");
    saveLog();
```

Кажется, что обе инструкции относятся к `if`. На самом деле только первая.

✓ Со скобками такой ошибки нет

```
if (temperature < 0)
{
    printf("Freezing\n");
    saveLog();
}
```

Несколько операторов `if`

Каждый оператор `if` проверяется независимо.

```
int grade = 50;

if (grade > 20)
{
    printf("Passed");
}

if (grade > 40)
{
    printf("Excellent");
}
```

Оба условия истинны, поэтому будут выполнены оба блока.

Когда нужен только один вариант

Иногда необходимо выбрать только одно действие.

Например:

- оценка может быть только одна;
- день недели может быть только один;
- ошибка может иметь только один код.

Оператор `else if`

```
if (grade > 40)
{
    printf("You got 5!");
}
else if (grade > 30)
{
    printf("You got 4!");
}
else if (grade > 20)
{
    printf("You got 3!");
}
```

После выполнения одной ветви остальные уже не проверяются.

Если использовать только `if`

```
if (grade > 40)
{
    printf("You got 5!");
}

if (grade > 30 && grade <= 40)
{
    printf("You got 4!");
}

if (grade > 20 && grade <= 30)
{
    printf("You got 3!");
}
```

Условия становятся сложнее, потому что каждый `if` проверяется независимо.

Используйте `else if`

✓ Проще читать

```
if (grade > 40)
{
    printf("You got 5!");
}
else if (grade > 30)
{
    printf("You got 4!");
}
else if (grade > 20)
{
    printf("You got 3!");
}
```

✗ Сложнее читать

```
if (grade > 40)
{
    printf("You got 5!");
}

if (grade > 30 && grade <= 40)
{
    printf("You got 4!");
}

if (grade > 20 && grade <= 30)
{
    printf("You got 3!");
}
```

Не путайте `=` и `==`

Присваивание

```
grade = 40;
```

Значение `40` записывается
в переменную `grade`.

`=` — записать значение в переменную.

`==` — сравнить два значения.

Сравнение

```
if (grade == 40)
{
    printf("Excellent");
}
```

Проверяется,
равно ли значение `40`.

Зачем нужны циклы?

Иногда одну и ту же операцию нужно выполнить несколько раз.

Например:

- вывести сообщение несколько раз;
- обработать несколько чисел;
- прочитывать несколько строк файла.

Без цикла

```
printf("Hello\n");  
printf("Hello\n");  
printf("Hello\n");  
printf("Hello\n");  
printf("Hello\n");
```

А если нужно вывести сообщение 1000 раз?

Цикл

Цикл — это конструкция, которая позволяет выполнять один и тот же блок кода несколько раз.

Цикл `while`

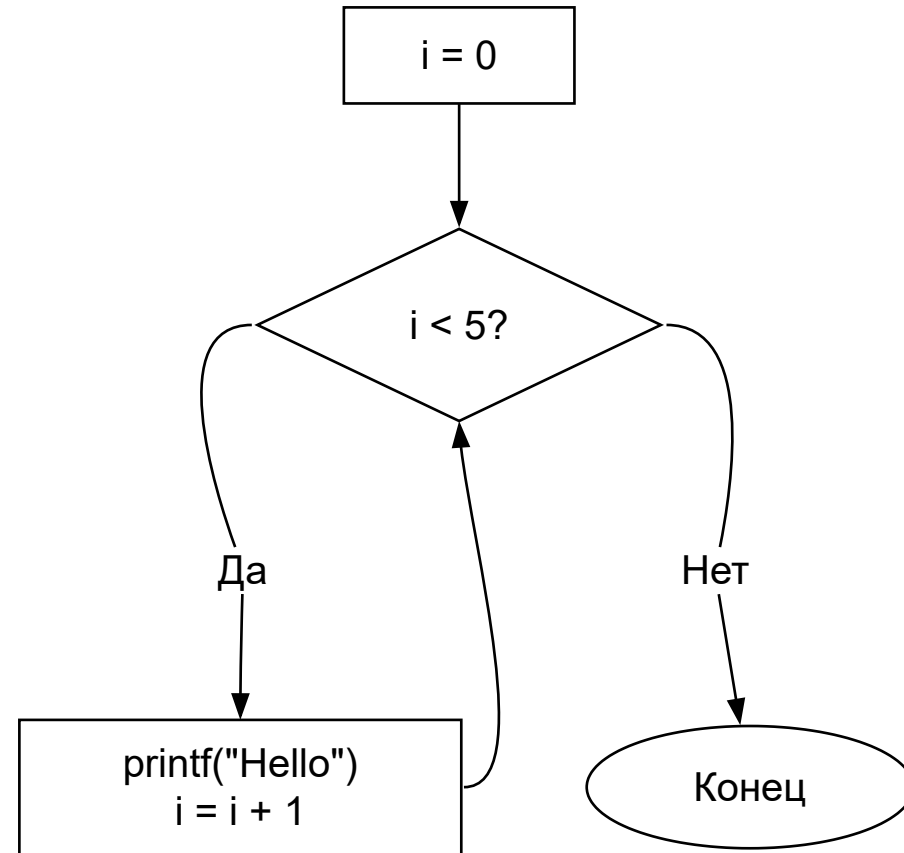
```
while (условие)
{
    инструкции;
}
```

Пока условие истинно, блок выполняется снова и снова.

Первый пример

```
int i = 0;

while (i < 5)
{
    printf("Hello\n");
    i = i + 1;
}
```



Значения переменной `i`

Начало

```
i = 0  
i = 1  
i = 2  
i = 3  
i = 4
```

После этого

```
i = 5
```

Условие `i < 5` становится ложным, и цикл заканчивается.

Что такое счётчик?

Счётчик — это переменная, значение которой изменяется на каждой итерации цикла.

В нашем примере счётчиком является переменная `i` .

Выполнение по шагам

```
i = 0
```

```
0 < 5 ?
```

```
Да
```

```
печать
```

```
Hello
```

```
i = 1
```

Затем цикл снова проверяет условие.

Что будет, если забыть изменить счётчик?

```
int i = 0;

while (i < 5)
{
    printf("Hello\n");
}
```

Переменная `i` никогда не изменяется. Условие всегда истинно.

Бесконечный цикл

```
while (1)
{
    printf("Hello\n");
}
```

Такой цикл никогда не закончится.

Иногда это необходимо, но чаще всего это ошибка.

Цикл может не выполняться ни разу

```
int i = 10;

while (i < 5)
{
    printf("Hello\n");
}
```

Начало

`i = 10`

Проверка

`10 < 5 ?`

Нет

Если условие сразу ложно, тело цикла не выполнится ни одного раза.

Условие может быть любым

Цикл `while` не обязательно использует счётчик.

```
while (temperature > 0)
{
    printf("Cooling...\n");
}
```

```
while (count != 0)
{
    printf("Processing...\n");
    count = count - 1;
}
```

Главное, чтобы условие можно было проверить.

Ещё один пример

```
int i = 1;

while (i <= 5)
{
    printf("%d\n", i);
    i = i + 1;
}
```

Код программы.

Вывод

```
1
2
3
4
5
```

Обратный счёт

```
int i = 5;

while (i > 0)
{
    printf("%d\n", i);
    i = i - 1;
}
```

Код программы.

Вывод

```
5
4
3
2
1
```


Осторожно с условием



```
int i = 0;

while (i < 5)
{
    printf("%d\n", i);
    i++;
}
```

Выводится 5 чисел.



```
int i = 0;

while (i <= 5)
{
    printf("%d\n", i);
    i++;
}
```

Выводится 6 чисел.

Разница всего в одном символе, но результат отличается.

Иногда одной проверки недостаточно

Мы уже знаем, что цикл `while` может не выполниться ни одного раза.

Но иногда действие нужно выполнить хотя бы один раз.

Например:

- показать меню;
- попросить пользователя ввести число;
- повторять ввод, пока он неверный.

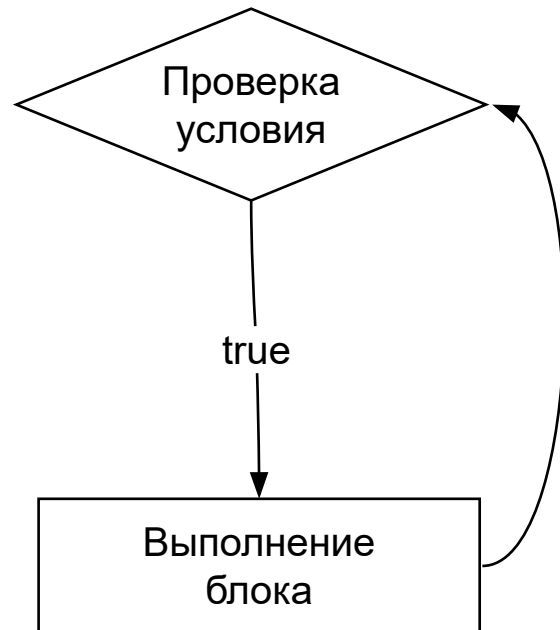
Цикл `do...while`

```
do
{
    инструкции;
}
while (условие);
```

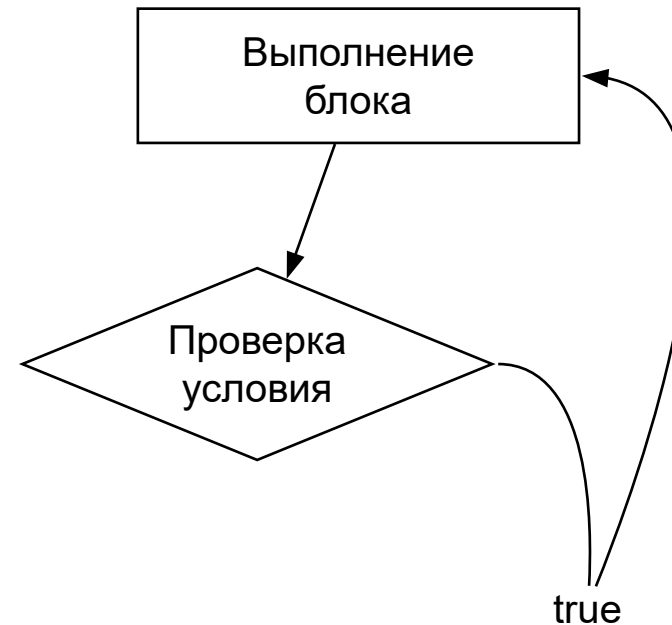
Сначала выполняется блок, затем проверяется условие.

while и do...while

while

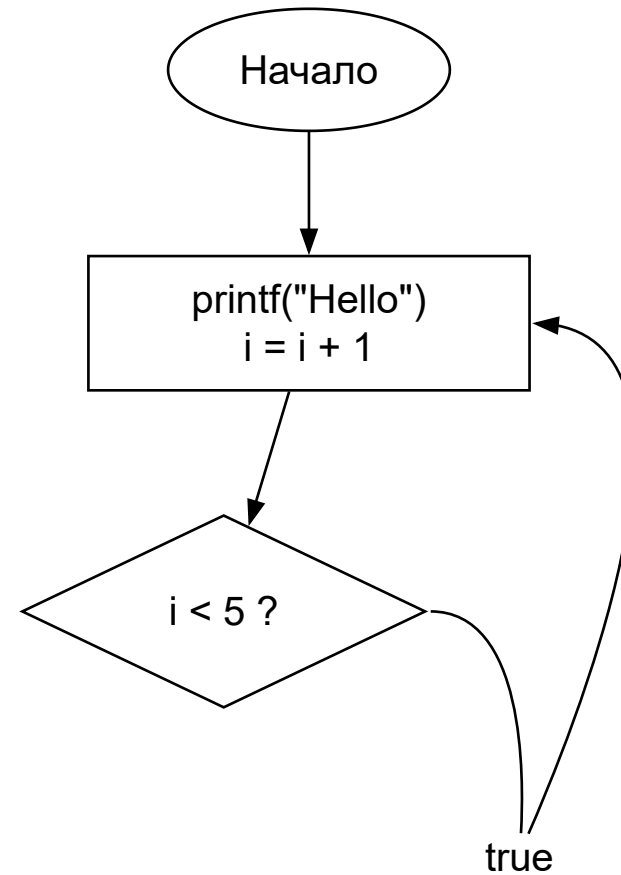


do...while



Первый пример

```
int i = 0;  
  
do  
{  
    printf("Hello\n");  
    i = i + 1;  
}  
while (i < 5);
```



Сравнение

while

```
int i = 10;

while (i < 5)
{
    printf("Hello\n");
}
```

Вывод

(ничего)

do...while

```
int i = 10;

do
{
    printf("Hello\n");
}
while (i < 5);
```

Вывод

Hello

Цикл `do...while` всегда выполняется хотя бы один раз.

Не забудьте точку с запятой

```
do  
{  
    ...  
}  
while (i < 5);
```

После `while (...)` обязательно ставится точка с запятой.

Зачем нужен цикл `for`?

Большинство циклов со счётчиком выглядят очень похоже.

```
int i = 0;

while (i < 5)
{
    printf("%d\n", i);
    i = i + 1;
}
```


Три части цикла

```
int i = 0;           // создание счётчика

while (i < 5)         // условие
{
    printf("%d\n", i);

    i = i + 1;        // изменение счётчика
}
```

Практически каждый такой цикл состоит из трёх частей.

Цикл `for`

```
for (создание; условие; изменение)
{
    инструкции;
}
```

Цикл `for` объединяет три части в одной строке.

Первый пример

```
for (int i = 0; i < 5; i = i + 1)
{
    printf("%d\n", i);
}
```

while и for

while

```
int i = 0;

while (i < 5)
{
    printf("%d\n", i);
    i = i + 1;
}
```

for

```
for (int i = 0; i < 5; i = i + 1)
{
    printf("%d\n", i);
}
```

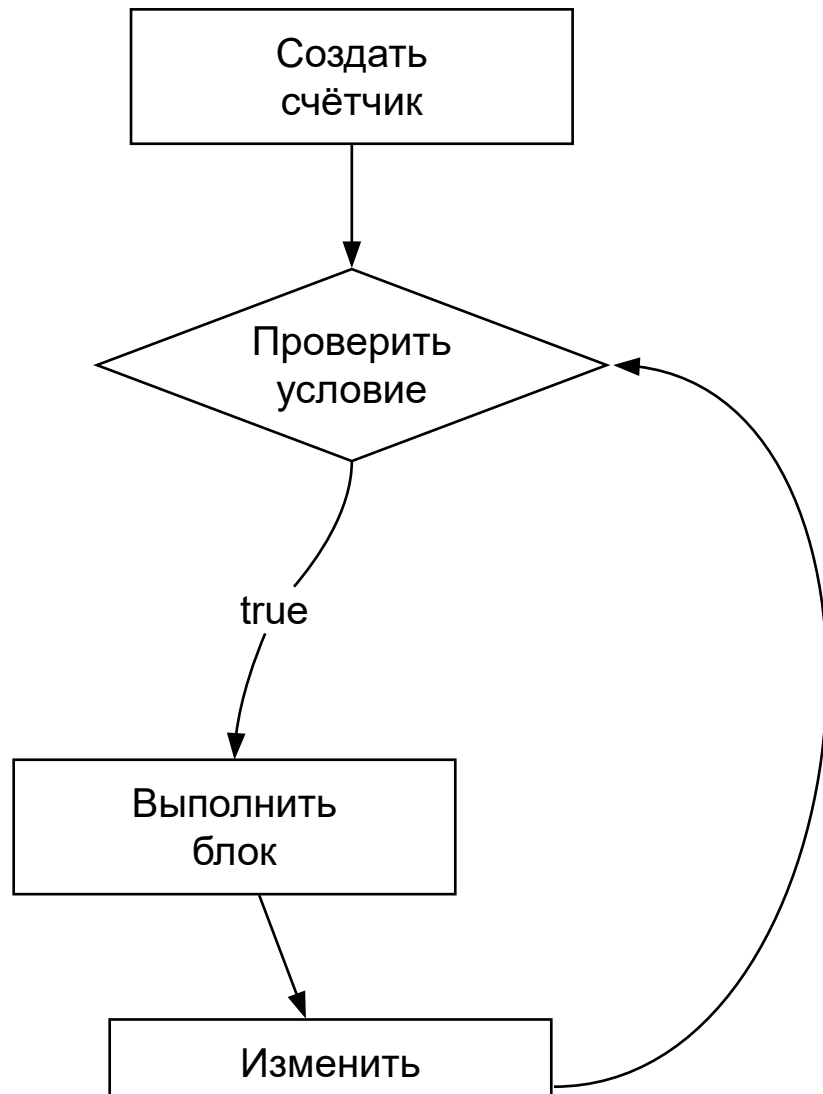
Оба цикла выполняют одну и ту же работу.

Из чего состоит **for**?

```
for (int i = 0; i < 5; i = i + 1)
```



Как работает цикл `for` ?



Пример

Код программы.

```
for (int i = 1; i <= 5; i = i + 1)
{
    printf("%d\n", i);
}
```

Вывод

```
1
2
3
4
5
```

Обратный счёт

Код программы.

```
for (int i = 5; i > 0; i = i - 1)
{
    printf("%d\n", i);
}
```

Вывод

```
5
4
3
2
1
```


Когда использовать `for`?

`while`

Используется, когда заранее неизвестно, сколько раз выполнится цикл.

`for`

Используется, когда есть счётчик, который изменяется на каждой итерации.

Во многих задачах цикл `for` оказывается самым удобным выбором.

Более короткая запись

В языке **C** увеличение переменной на единицу можно записать короче.

```
i = i + 1;
```

Эквивалентная запись:

```
i++;
```

Теперь цикл выглядит короче

```
for (int i = 0; i < 5; i++)  
{  
    printf("%d\n", i);  
}
```

Записи `i = i + 1` и `i++` выполняют одно и то же действие.

Управляющие конструкции можно комбинировать

Конструкции языка `C` можно вкладывать друг в друга.

Например:

- внутри `if` может быть цикл;
- внутри цикла может быть `if` ;
- внутри одного цикла может находиться другой цикл.

Такая запись называется вложением конструкций.

if внутри цикла

```
for (int i = 1; i <= 10; i++)  
{  
    if (i < 5)  
    {  
        printf("%d\n", i);  
    }  
}
```

На каждой итерации цикла дополнительно проверяется условие.

Цикл внутри цикла

Код программы.

```
for (int i = 1; i <= 3; i++)  
{  
    for (int j = 1; j <= 2; j++)  
    {  
        printf("*");  
    }  
    printf("\n");  
}
```

Вывод

```
**  
**  
**
```

Можно комбинировать любые конструкции

```
while (running)
{
    if (temperature > 100)
    {
        for (int i = 0; i < 3; i++)
        {
            printf("Warning!\n");
        }
    }
}
```

Нет строгих ограничений на вложение конструкций. Главное — чтобы код оставался понятным и читаемым.

Оператор switch

Если необходимо сравнить одно выражение с несколькими значениями, вместо длинной цепочки `if ... else if` можно использовать оператор `switch`.

Общий вид switch

```
switch (выражение)
{
  case значение1:
    операторы;
    break;

  case значение2:
    операторы;
    break;

  default:
    операторы;
}
```

Пример switch

```
int day = 3;

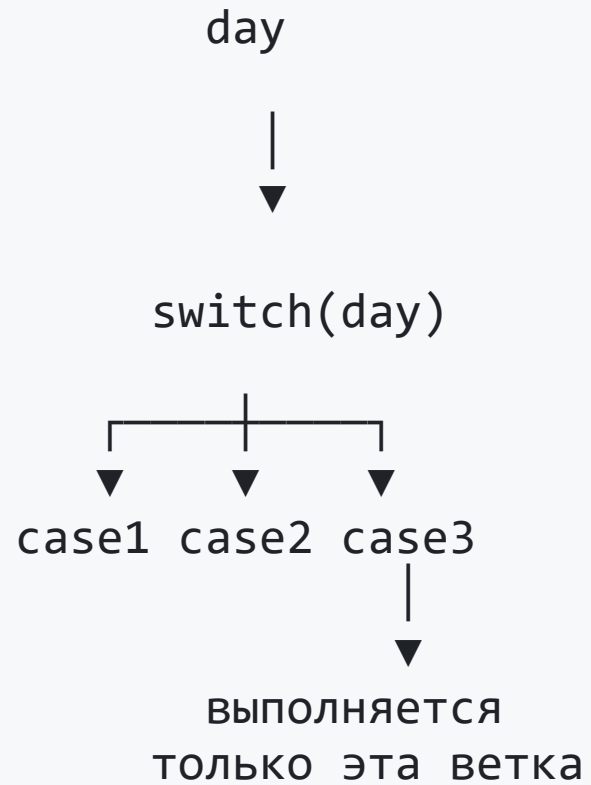
switch (day)
{
case 1:
    printf("Monday\n");
    break;

case 2:
    printf("Tuesday\n");
    break;

case 3:
    printf("Wednesday\n");
    break;

default:
    printf("Unknown\n");
}
```

Как работает switch?



Для чего нужен break?

```
switch (day)
{
case 1:
    printf("Monday\n");
    break;

case 2:
    printf("Tuesday\n");
    break;
}
```

break завершает выполнение **switch** .

Что произойдёт без break?

```
int x = 1;

switch (x)
{
case 1:
    printf("One\n");

case 2:
    printf("Two\n");
}
```

Что будет выведено?

Ответ

```
One  
Two
```

После выполнения подходящего `case` программа продолжает выполнять следующие ветви, пока не встретит `break` или конец `switch`.

Такое поведение называется проваливанием (*fall-through*).

Несколько case

Иногда проваливание используют специально.

```
switch (month)
{
case 12:
case 1:
case 2:
    printf("Winter\n");
    break;
}
```

Ветка `default`

```
switch (command)
{
case 1:
    ...
    break;

case 2:
    ...
    break;

default:
    printf("Unknown command\n");
}
```

Ветка `default` выполняется, если не подошёл ни один `case` .

Оператор break

Инструкция

```
break;
```

немедленно завершает выполнение

- цикла;
- оператора `switch` .

break в цикле

```
while (1)
{
    scanf("%d", &x);

    if (x == 0)
        break;

    printf("%d\n", x);
}
```

Цикл завершится, когда пользователь введёт 0 .

continue

Инструкция

```
continue;
```

прерывает выполнение текущей итерации цикла и сразу переходит к следующей.

Пример continue

Код программы

```
for (int i = 1; i <= 5; i++)  
{  
    if (i == 3)  
        continue;  
  
    printf("%d ", i);  
}
```

Вывод

1 2 4 5

break и continue

break	continue
завершает цикл	переходит к следующей итерации
цикл заканчивается	цикл продолжается

Оператор goto

Инструкция

```
goto метка;
```

передаёт управление на указанную метку программы.

Пример goto

Код программы

```
printf("Start\n");  
  
goto end;  
  
printf("Never\n");  
  
end:  
  
printf("Finish\n");
```

Вывод

```
Start  
Finish
```

Метка

Метка имеет вид

```
name:
```

Например

```
error: , finish: , retry:
```

После выполнения

```
goto error;
```

программа продолжит работу с этой метки.

Стоит ли использовать goto?

Всегда программу можно написать без `goto`.

Обычно используются

- `if`;
- `switch`;
- циклы;
- функции.

Когда goto бывает полезен?

Например, при обработке ошибок и *освобождении ресурсов*.

```
FILE *a = fopen(...);  
FILE *b = fopen(...);
```

```
if (a == NULL)  
    goto error;
```

```
if (b == NULL)  
    goto error;
```

```
/* работа */
```

```
error:
```

```
if (b) fclose(b);  
if (a) fclose(a);
```